

WEEK 2 PRACTICAL ACTIVITY

TASK 1 — IDENTIFY THE DEVELOPER

Evidence

Screenshot 1

```
● PS C:\Users\nelsh\Desktop\GIT\my-website> pwd

Path
----
C:\Users\nelsh\Desktop\GIT\my-website

● PS C:\Users\nelsh\Desktop\GIT\my-website> git config --global user.name
nellsha
● PS C:\Users\nelsh\Desktop\GIT\my-website> git config --global user.email
nelshashrestha123@gmail.com
○ PS C:\Users\nelsh\Desktop\GIT\my-website> █
```

TASK 2 — START VERSION CONTROL

Evidence

Screenshot 2

```
● PS C:\Users\nelsh\Desktop\GIT\my-website> git init
Initialized empty Git repository in C:/Users/nelsh/Desktop/GIT/my-website/.git/
● PS C:\Users\nelsh\Desktop\GIT\my-website> git branch -M master
● PS C:\Users\nelsh\Desktop\GIT\my-website> git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    index.html
    styles.css

nothing added to commit but untracked files present (use "git add" to track)
○ PS C:\Users\nelsh\Desktop\GIT\my-website> █
```

TASK 3 — CREATE THE FIRST VERSION

Evidence

Screenshot 3

```
• PS C:\Users\nelsh\Desktop\GIT\my-website> git add index.html
warning: in the working copy of 'index.html', LF will be replaced by CRLF the next time Git touches it
• PS C:\Users\nelsh\Desktop\GIT\my-website> git status

On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   index.html

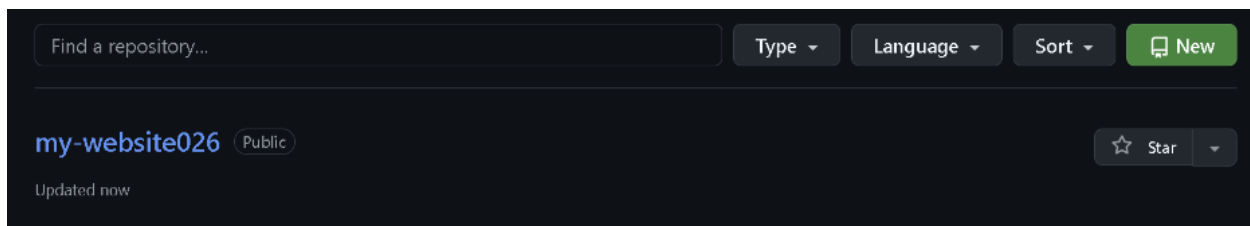
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        styles.css

• PS C:\Users\nelsh\Desktop\GIT\my-website> git commit -m "Add initial HTML version"
[master (root-commit) fc0a753] Add initial HTML version
 1 file changed, 36 insertions(+)
 create mode 100644 index.html
• PS C:\Users\nelsh\Desktop\GIT\my-website> git log --oneline
fc0a753 (HEAD -> master) Add initial HTML version
○ PS C:\Users\nelsh\Desktop\GIT\my-website> █
```

TASK 4 — CREATE THE ONLINE REPOSITORY

Evidence

Screenshot 4



TASK 5 — CONNECT LOCAL GIT TO GITHUB

Evidence

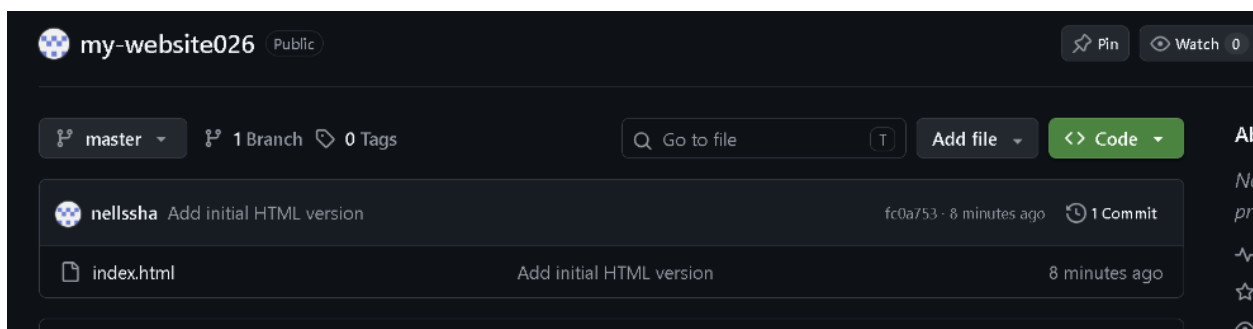
Screenshot 5

```
• PS C:\Users\nelsh\Desktop\GIT\my-website> git remote add origin https://github.com/nellssha/my-website026.git
• PS C:\Users\nelsh\Desktop\GIT\my-website> git remote -v
origin https://github.com/nellssha/my-website026.git (fetch)
origin https://github.com/nellssha/my-website026.git (push)
○ PS C:\Users\nelsh\Desktop\GIT\my-website> █
```

TASK 6 — PUBLISH THE FIRST VERSION

Evidence

Screenshot 6



TASK 7 — DEVELOP THE CSS SAFELY

Evidence

Your evidence must show:

- master branch
- add-css branch
- add-css as your current branch
- CSS work recorded in Git

Screenshot 7

```

● PS C:\Users\nelsh\Desktop\GIT\my-website> git push -u origin master
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 16 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 740 bytes | 740.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/nellssha/my-website026.git
 * [new branch]      master -> master
branch 'master' set up to track 'origin/master'.
● PS C:\Users\nelsh\Desktop\GIT\my-website> git switch -c add-css
Switched to a new branch 'add-css'
● PS C:\Users\nelsh\Desktop\GIT\my-website> git branch
* add-css
  master
● PS C:\Users\nelsh\Desktop\GIT\my-website> git status
On branch add-css
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        styles.css

nothing added to commit but untracked files present (use "git add" to track)
● PS C:\Users\nelsh\Desktop\GIT\my-website> git add style.css
fatal: pathspec 'style.css' did not match any files
● PS C:\Users\nelsh\Desktop\GIT\my-website> git add styles.css
● PS C:\Users\nelsh\Desktop\GIT\my-website> git status
On branch add-css
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   styles.css

● PS C:\Users\nelsh\Desktop\GIT\my-website> git commit -m "Add CSS styling"
[add-css 3699c82] Add CSS styling
1 file changed, 70 insertions(+)
create mode 100644 styles.css
● PS C:\Users\nelsh\Desktop\GIT\my-website> git log --oneline
3699c82 (HEAD -> add-css) Add CSS styling
fc0a753 (origin/master, master) Add initial HTML version
● PS C:\Users\nelsh\Desktop\GIT\my-website> git branch
* add-css
  master

```

TASK 8 — INTEGRATE THE CSS FEATURE

Evidence

Show evidence that:

1. The CSS work was committed.

2. The add-css branch has been integrated into master.

Screenshot 8

```
● PS C:\Users\nelsh\Desktop\GIT\my-website> git commit -m "Add CSS styling"
[add-css 3699c82] Add CSS styling
1 file changed, 70 insertions(+)
create mode 100644 styles.css
● PS C:\Users\nelsh\Desktop\GIT\my-website> git log --oneline
3699c82 (HEAD -> add-css) Add CSS styling
fc0a753 (origin/master, master) Add initial HTML version
● PS C:\Users\nelsh\Desktop\GIT\my-website> git branch
* add-css
  master
● PS C:\Users\nelsh\Desktop\GIT\my-website> git switch master
Switched to branch 'master'
Your branch is up to date with 'origin/master'.
● PS C:\Users\nelsh\Desktop\GIT\my-website> git branch
  add-css
* master
● PS C:\Users\nelsh\Desktop\GIT\my-website> git merge add-css
Updating fc0a753..3699c82
Fast-forward
 styles.css | 70 +++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
1 file changed, 70 insertions(+)
create mode 100644 styles.css
● PS C:\Users\nelsh\Desktop\GIT\my-website> git log --oneline --graph --all
* 3699c82 (HEAD -> master, add-css) Add CSS styling
* fc0a753 (origin/master) Add initial HTML version
● PS C:\Users\nelsh\Desktop\GIT\my-website> git status
On branch master
Your branch is ahead of 'origin/master' by 1 commit.
(use "git push" to publish your local commits)
```

TASK 9 — PUBLISH THE FINAL WEBSITE

Evidence

1. Your GitHub repository URL
2. Screenshot showing both files on GitHub
3. Screenshot showing your final Git history

Screenshot 9

```

● PS C:\Users\nelsh\Desktop\GIT\my-website> git push origin master
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 16 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 658 bytes | 658.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/nellssha/my-website026.git
   fc0a753..3699c82  master -> master
● PS C:\Users\nelsh\Desktop\GIT\my-website> git log --oneline --graph --all
* 3699c82 (HEAD -> master, origin/master, add-css) Add CSS styling
* fc0a753 Add initial HTML version
○ PS C:\Users\nelsh\Desktop\GIT\my-website>

```

Q1. What is the purpose of Git in this practical?

Git is used to track changes to the website and maintain its development history. It allows different versions of the project to be recorded so that changes can be managed safely.

Q2. What is the difference between the Working Area and the Staging Area?

The Working Area contains the files that I am currently creating or modifying. The Staging Area contains the specific changes that I have selected to include in the next commit.

Q3. What is the difference between `git add` and `git commit`?

`git add` moves selected changes from the Working Area to the Staging Area. `git commit` then saves those staged changes as a permanent point in the Git project's history.

Q4. Why did we create the `add-css` branch instead of working directly on `master`?

We created the `add-css` branch so that the CSS changes could be developed separately without changing the main version of the website. After the CSS work was completed and reviewed, it could be merged into `master`.

Q5. What is the difference between `git commit` and `git push`?

`git commit` saves changes to the local Git repository, while `git push` sends those committed changes from the local repository to the remote GitHub repository.

PART B — COLLABORATIVE GITHUB WORKFLOW In

Part A, you created branches and merged changes locally.

In Part B, you will learn how a development team can use **GitHub branches and Pull Requests** to develop and integrate a new feature.

Important

In this activity, the feature-branch must be created **on GitHub first**.

You will then:

```
graph TD; A[Create branch on GitHub] --> B[Fetch branch information]; B --> C[Create local branch from remote branch]; C --> D[Work on the feature]; D --> E[Commit the feature]; E --> F[Push feature branch to GitHub]; F --> G[Create Pull Request]; G --> H[Review the Pull Request]; H --> I[Merge into main]; I --> J[Verify the final result];
```

For this individual practical, you will **simulate the review stage** yourself. In a real development team, another developer or Team Leader would normally perform the review.

TASK 10 — CREATE feature-branch ON GITHUB

Scenario

Your Team Leader says:

"Create a new development branch directly on GitHub. We will use this branch to develop a new website feature."

Your task

Open your GitHub repository:

my-website026

Using the GitHub **branch dropdown**, create a new branch named:

feature-branch

Create the branch from the current:

main

or:

master

Important

For this task:

- The branch must be created **on GitHub first**.
- Do **not** create feature-branch locally before creating it on GitHub.

Expected result

Your GitHub repository should now contain:

```
main/master
|
└─ feature-branch
```

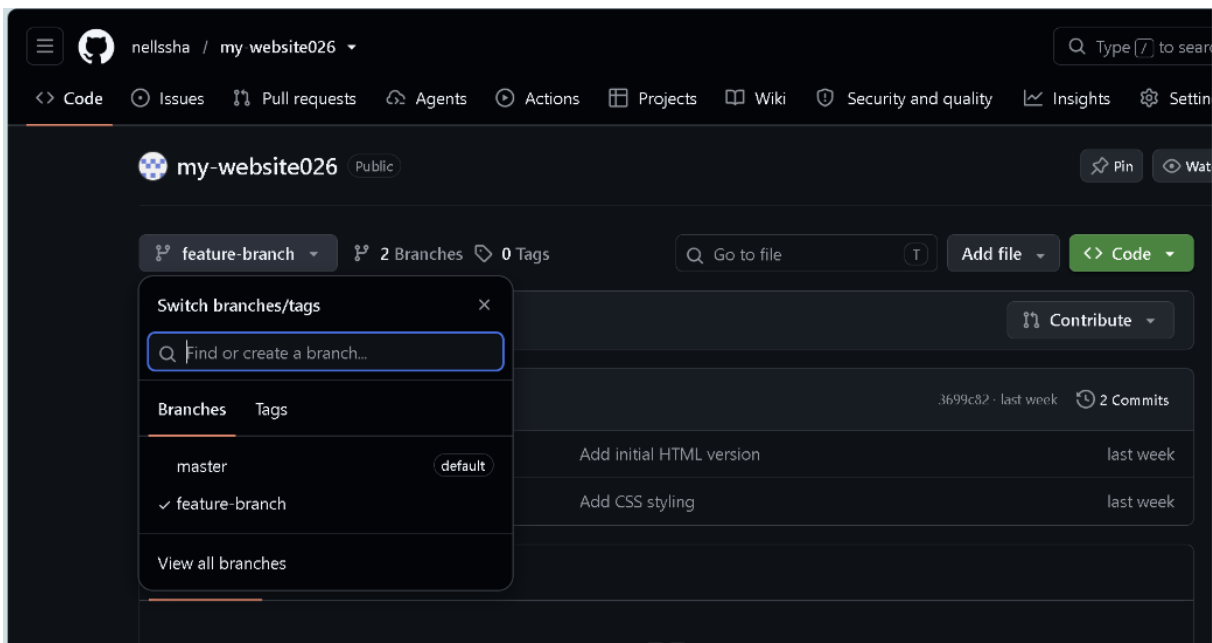
At this point, feature-branch exists on **GitHub**, but you have not yet created a local feature-branch on your computer.

Evidence

Provide a screenshot showing:

- Your GitHub repository
- feature-branch
- feature-branch existing on GitHub

Screenshot 10



TASK 11 — FETCH THE REMOTE BRANCH INFORMATION

Scenario

The feature-branch now exists on GitHub.

However, your local Git repository does not yet know about this newly created remote branch.

Your Team Leader says:

"Update your local Git repository with the latest information from GitHub."

Your task

Use Git to retrieve the latest information from the remote repository.

Then check the available branches.

You should be able to identify the remote branch as:

origin/feature-branch

Important concept

At this stage:

origin/feature-branch

represents the branch that exists on GitHub.

3

It is **not yet your local working branch**.

Expected result

Your local Git should show something similar to:

```
main
remotes/origin/main
remotes/origin/feature-branch
```

Evidence

Provide a screenshot showing:

- The Git operation used to retrieve the remote information
- origin/feature-branch visible as a remote branch

Screenshot 11

```
● PS C:\Users\nelsh\Desktop\GIT\my-website> git fetch origin
From https://github.com/nellssha/my-website026
* [new branch]      feature-branch -> origin/feature-branch
● PS C:\Users\nelsh\Desktop\GIT\my-website> git branch -a
add-css
* master
remotes/origin/HEAD -> origin/master
remotes/origin/feature-branch
remotes/origin/master
● PS C:\Users\nelsh\Desktop\GIT\my-website> git branch -a
add-css
* master
remotes/origin/HEAD -> origin/master
remotes/origin/feature-branch
remotes/origin/master
○ PS C:\Users\nelsh\Desktop\GIT\my-website> █
```

TASK 12 — CREATE AND SWITCH TO THE LOCAL feature-branch

Scenario

Your local Git can now see the remote branch:

origin/feature-branch

Your Team Leader says:

"Create a local working branch from the remote feature-branch and switch to it."

Your task

Create a local branch named:

feature-branch

based on:

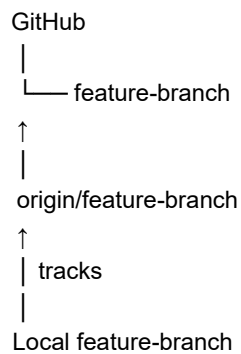
origin/feature-branch

Then switch to the local feature-branch .

Important concept

You are now creating the **local working branch**.

The relationship is:



Your local feature-branch should track the remote:

origin/feature-branch

Verify

Check that:

- feature-branch is your current local branch. • main or master also exists.
- Your local feature-branch tracks origin/feature-branch . **Evidence**

Provide a screenshot showing:

- feature-branch
- main or master

- feature-branch as the current branch •
- The tracking relationship with origin/feature-branch

Screenshot 12

```
PS C:\Users\nelsh\Desktop\GIT\my-website> git switch -c feature-branch --track origin/feature-branch
Switched to a new branch 'feature-branch'
PS C:\Users\nelsh\Desktop\GIT\my-website> git branch -vv
  add-css          3699c82 Add CSS styling
* feature-branch  3699c82 [origin/feature-branch] Add CSS styling
  master          3699c82 [origin/master] Add CSS styling
PS C:\Users\nelsh\Desktop\GIT\my-website>
```

5

TASK 13 — DEVELOP A SMALL FEATURE

Scenario

Your Team Leader gives you the following requirement:

"Add one small improvement to the website."

Your task

While working on the local:

feature-branch

make **one small change** to the website.

You may:

- Add a footer
- Add a heading
- Add a paragraph
- Add your name

• •

Modify website text

- Add a simple CSS improvement

-

- Add another small HTML element

For example:

```
<footer>
```

```
Developed by Your Name - 2026
```

```
</footer>
```

Important

Do not redesign the entire website.

The purpose of this task is to demonstrate the complete:

Branch → Develop → Commit → Push → Pull Request → Merge workflow.

6

TASK 14 — RECORD THE FEATURE IN LOCAL GIT

Scenario

Your feature has been completed on your computer.

Your Team Leader says:

"Record your completed feature in Git so that it becomes part of the project's development history."

Your task

Use the appropriate Git operations to:

1. Check the status of your repository.

2. Stage the file or files you changed.
3. Create a meaningful commit.
4. Verify that the commit exists in your Git history.

Important

The commit should be made while you are working on:

feature-branch

Evidence

Provide a screenshot showing:

- feature-branch as the current branch •
Your feature commit
- Your meaningful commit message

Screenshot 13

```

● PS C:\Users\nelsh\Desktop\GIT\my-website> git branch
  add-css
  * feature-branch
  master
● PS C:\Users\nelsh\Desktop\GIT\my-website> git status
On branch feature-branch
Your branch is up to date with 'origin/feature-branch'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   index.html

no changes added to commit (use "git add" and/or "git commit -a")
● PS C:\Users\nelsh\Desktop\GIT\my-website> git add index.html
warning: in the working copy of 'index.html', LF will be replaced by CRLF the next time Git touches it
● PS C:\Users\nelsh\Desktop\GIT\my-website> git add index.html
● PS C:\Users\nelsh\Desktop\GIT\my-website> git status
On branch feature-branch
Your branch is up to date with 'origin/feature-branch'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   index.html

● PS C:\Users\nelsh\Desktop\GIT\my-website> git commit -m "Add website footer"
[feature-branch 19c4b2d] Add website footer
 1 file changed, 3 insertions(+)
● PS C:\Users\nelsh\Desktop\GIT\my-website> git log --oneline --graph --all
* 19c4b2d (HEAD -> feature-branch) Add website footer
* 3699c82 (origin/master, origin/feature-branch, origin/HEAD, master, add-css) Add CSS styling
* fc0a753 Add initial HTML version
○ PS C:\Users\nelsh\Desktop\GIT\my-website> 

```

TASK 15 — PUSH THE FEATURE BRANCH TO GITHUB

Scenario

Your feature has been committed locally.

However, the new feature is still only on your computer.

7

Your Team Leader says:

"Publish your feature branch to GitHub so that the feature can be reviewed."

Your task

Push your local:

feature-branch

to the corresponding branch on GitHub.

Then open GitHub and verify that:

• •

feature-branch exists

Your latest commit is visible

•

Your new feature is present on the branch

Important concept

Before the push:

Your computer
└─ feature-branch
└─ New feature

After the push:

Your computer
└─ feature-branch
└─ New feature
|
| PUSH
↓
GitHub
└─ feature-branch
└─ New feature

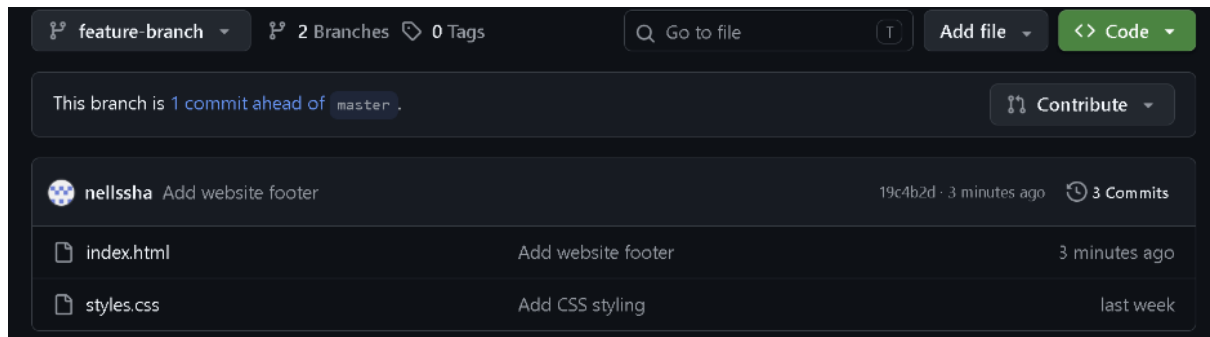
The feature is now on GitHub, but it has **not yet been merged into main/master** . 8

Evidence

Provide a screenshot showing:

- feature-branch on GitHub •
Your new feature/change
- Your latest commit

Screenshot 14



TASK 16 — CREATE A PULL REQUEST

Scenario

Your feature is now available on GitHub.

In a real development team, the feature would normally be reviewed before being added to the main project.

Your Team Leader says:

"Create a Pull Request so that the feature can be reviewed before it is merged into the main branch."

Your task

On GitHub:

1. Open the **Pull Requests** section.
2. Select **New Pull Request**.
3. Set the **base branch** to:

main

or:

master

1. Set the **comparison branch** to:

feature-branch

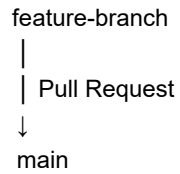
1. 2.

Review the changes shown by GitHub.

Create the Pull Request.

Understand the direction

The Pull Request means:



You are saying:

"My feature is ready. Please review these changes and consider adding them to the main branch."

Important

A Pull Request is **not a Git command**.

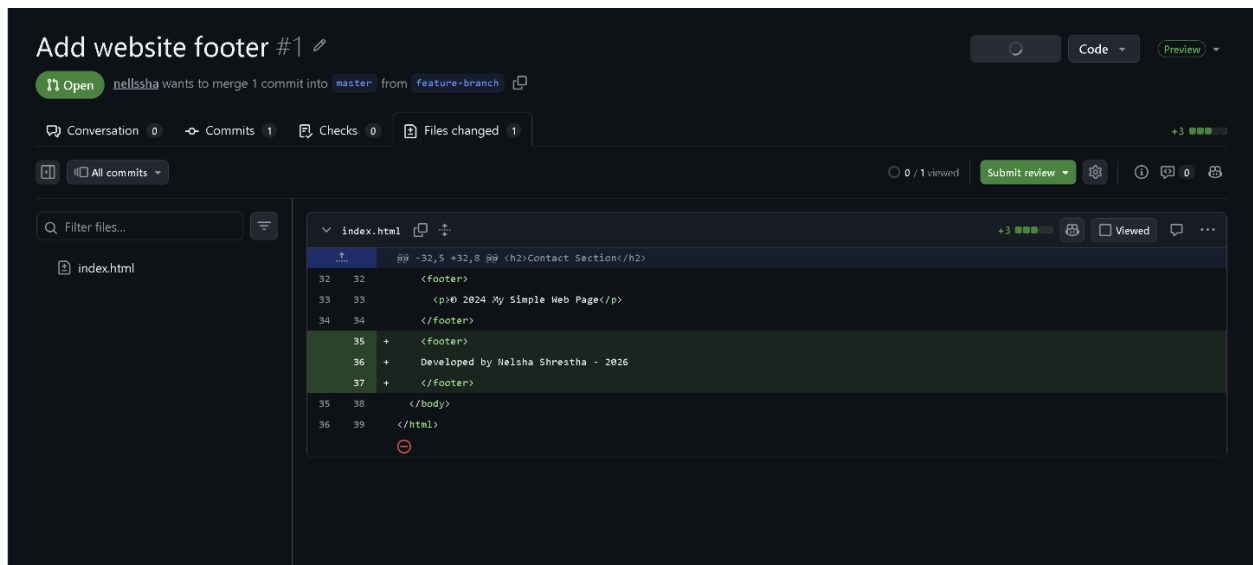
It is a **GitHub feature used to review and merge changes between branches**.

Evidence

Provide a screenshot showing:

- The Pull Request
- • Base branch
- Comparison branch
- Proposed changes

Screenshot 15



TASK 17 — REVIEW AND MERGE THE PULL REQUEST

Scenario

In a real development team, another developer or Team Leader would review your Pull Request. 10

Because this is an individual practical, **you will simulate the review process yourself.**

Your task

On GitHub:

1. Open your Pull Request.
2. 3.

Review the proposed changes.

Check that the feature meets the requirement.

4. Confirm that no unwanted changes have been introduced.

5. Approve the Pull Request.

6. Merge the Pull Request.

Confirm the merge if GitHub asks for confirmation.

Expected result

The changes from:

feature-branch

should now be integrated into:

main

or:

master

The workflow is:

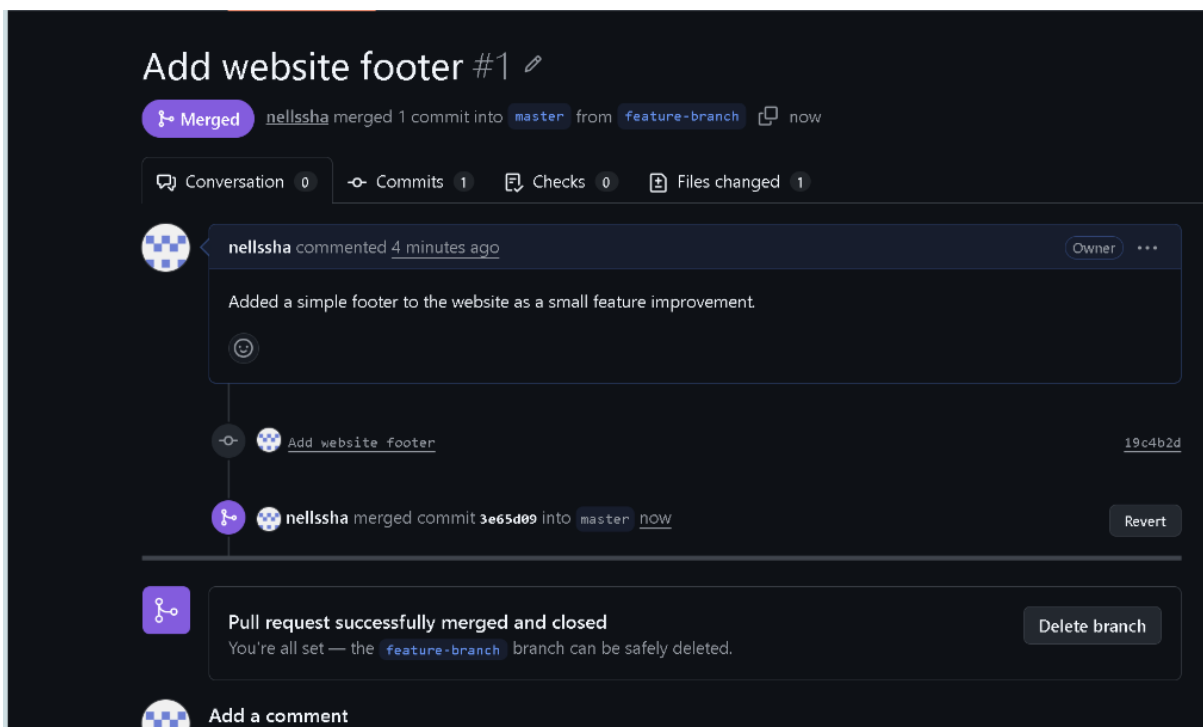
```
graph TD
    A[feature-branch] --> B[Pull Request]
    B --> C[Review]
    C --> D[Approve]
    D --> E[Merge]
    E --> F[main/master]
```

Evidence

Provide a screenshot showing:

- The Pull Request has been reviewed
- The Pull Request has been approved
- The Pull Request has been successfully merged

Screenshot 16



TASK 18 — VERIFY THE FINAL RESULT

Scenario

The feature has now been merged.

Your Team Leader says:

"Verify that the approved feature is now part of the main version of the website."

Your task

Open the:

main

or:

master

branch on GitHub.

Verify that:

• • •
index.html exists style.css exists

Your new feature/change exists

•
The Pull Request was successfully merged 12

Expected final repository

my-website026

```
|
|
|— index.html
|— style.css
```

Your new feature should also be visible in the appropriate file.

Final workflow

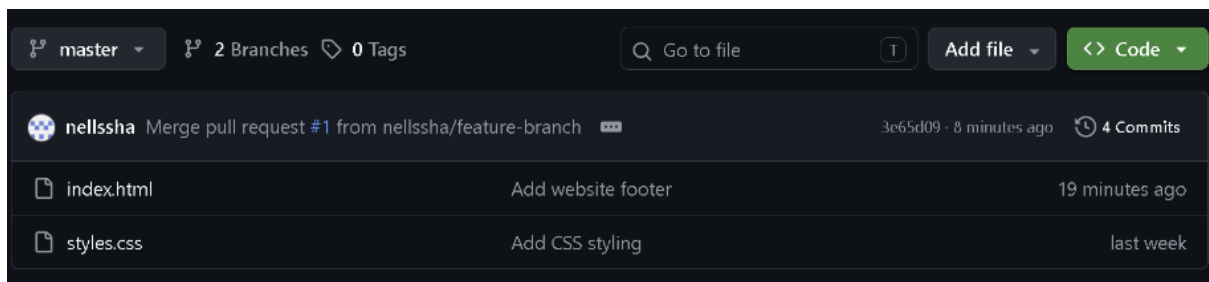
```
GitHub
|
| Create feature-branch
↓
Remote feature-branch
|
| Fetch
↓
Local Git
|
| Create local tracking branch
↓
Local feature-branch
|
| Develop
↓
Commit
|
| Push
↓
GitHub feature-branch
|
| Pull Request
↓
Review
|
| Approve
↓
Merge
↓
main/master
```

Evidence

Provide:

1. Screenshot of the final main/master branch
2. Screenshot showing your feature in the final version
3. Screenshot showing the merged Pull Request

Screenshot 17



THE KEY CONCEPTS YOU MUST UNDERSTAND

Before completing Part B, make sure you understand the difference between these operations:

Operation	What it means
Create branch on GitHub	Create the development branch on the remote repository
Fetch	Retrieve information about the remote branch
Create local branch	Create a local working branch based on the remote branch
Edit	Actually develop/change the website
Add	Prepare your changed file(s) for a commit
Commit	Record your changes in local Git history

Push Send your committed changes to GitHub
Pull Request Ask for the feature branch to be reviewed and merged
Review Check the proposed changes
Approve Accept the proposed changes
Merge Integrate the approved changes into main/master

Remember

FETCH

GitHub → Local Git

"Tell me what exists."

PUSH

Local Git → GitHub

"Send my committed work."

PULL REQUEST

14

feature-branch → main/master

"Please review my work before merging it."

Do not confuse git pull with a Pull Request. They are different concepts. 15

